Tight Security Proof of Kylin

Based on "Recovering the Tight Security Proof of SPHINCS+"
by Hülsing and Kudinov (2022)

June 23, 2026

Abstract

We present a tight security proof for Kylin, a stateless hash-based signature scheme that improves upon SPHINCS+ by replacing the WOTS+ one-time signature scheme with CS-WOTS+ (Constant Sum WOTS+) and allowing diverse layer heights in the hypertree. Our proof follows the tight proof framework of Hülsing and Kudinov [1], which recovered the tight reduction for SPHINCS+ after the flaw identified by Kudinov, Kiktenko, and Fedorov [3]. The key technical contribution is the adaptation of the WOTS-TW security proof [1, Theorem 1] to CS-WOTS-TW, replacing the checksum-based encoding with the antichain-based CS-WOTS+ encoding. We show that all game transitions (PRF replacement, SM-UD hybrid, SM-TCR placement, SM-PRE extraction) carry over unchanged, with only the final preimage extraction step relying on the antichain property instead of the checksum.

Contents

1 Introduction **2**

2 Preliminaries **2**
2.1 Tweakable Hash Functions 2
2.2 Security Notions for Tweakable Hash Functions 3
2.3 Pseudorandom Functions 4
2.4 Interleaved Target Subset Resilience 4
2.5 EU-naCMA Security Model 4

3 CS-WOTS+: Constant Sum WOTS+ **4**
3.1 The Poset and Antichain 4
3.2 Cardinality and Recurrence 5
3.3 Encoding and Decoding 5
3.4 CS-WOTS-TW Scheme 6

4 Tight Security of CS-WOTS-TW (Single Instance) **6**

5 Multi-Instance CS-WOTS-TW **9**

6 The Kylin Construction **10**
6.1 Parameters 10
6.2 Hypertree with Diverse Layer Heights 10
6.3 FORS and Full Kylin 10
6.4 Signature Size 11

7 Security Reduction for Kylin **11**

8 Concrete Security Against Generic Attacks **13**

9 Comparison with the Tight SPHINCS+ Proof 14

1 Introduction

Kylin [4] is a stateless hash-based digital signature scheme submitted to the NGCC competition. It modifies SPHINCS+ [2] in two principal ways:

1. The one-time signature scheme is replaced from WOTS+ to **CS-WOTS+** (Constant Sum WOTS+), which uses a size-optimal constant-sum encoding based on the maximum antichain of the hash-chain poset. All codewords are pairwise incomparable, eliminating the explicit checksum.
2. The hypertree layers are allowed to have non-uniform heights $(h_0, \ldots, h_{d-1})$ with $\sum h_i = h$.

The original SPHINCS+ security proof [2] was shown to have a flaw by Kudinov, Kiktenko, and Fedorov [3]. Hülsing and Kudinov [1] recovered the tight security proof by formalizing the WOTS variant as WOTS-TW and proving its security in the EU-naCMA model using three properties of tweakable hash functions: SM-TCR, SM-PRE, and SM-UD. Since Kylin inherits all other components (hypertree, FORS, tweakable hash function framework, PRF-based key derivation) from SPHINCS+, its security proof can be obtained by adapting the tight proof of [1] to use CS-WOTS+ encoding in place of the checksum.

Our contribution. We provide a complete, self-contained tight security reduction for Kylin. The proof structure closely follows [1]:

- **Section 2** defines the cryptographic primitives and security notions (SM-TCR, SM-PRE, SM-UD, SM-DSPR for tweakable hash functions; EU-naCMA for signatures).
- **Section 3** defines CS-WOTS+, proves the antichain property, and establishes the encoding scheme.
- **Section 4** proves single-instance EU-naCMA security of CS-WOTS-TW (Theorem 4.1).
- **Section 5** extends to multi-instance security (Theorem 5.1).
- **Section 6** describes the Kylin construction, including diverse layer heights.
- **Section 7** gives the main EUF-CMA security reduction for Kylin (Theorem 7.1).
- **Section 8** derives concrete classical and quantum security bounds against generic attacks.

2 Preliminaries

2.1 Tweakable Hash Functions

Throughout this note, n denotes the Kylin security parameter in bytes, matching the specification and implementation. We write $N = 8n$ for the corresponding bit length. A tweakable hash function [2] takes public parameters $P \in \mathcal{P}$, a tweak $T \in \mathcal{T}$, and a message M, and outputs an N-bit hash value:

$$\mathsf{Th} : \mathcal{P} \times \mathcal{T} \times \{0,1\}^* \to \{0,1\}^N, \qquad \mathsf{Th}(P, T, M) \text{ or } \mathsf{Th}_{P,T}(M).$$

In Kylin (as in SPHINCS+), the public parameter $P = \mathsf{pk.seed}$ is an n-byte public seed, and the tweak T is an address $\mathsf{ADRS} \in \{0,1\}^{256}$ identifying the position of the hash call within the virtual structure. A *collection* of tweakable hash functions $\mathbf{Th} = \{\mathsf{Th}_{m_i}\}$ consists of several THFs with different message lengths m_i sharing the same public parameter P.

For Kylin we define the following THFs within the collection:

$$\mathsf{F} = \mathsf{Th}_1 : \{0,1\}^N \times \{0,1\}^{256} \times \{0,1\}^N \to \{0,1\}^N,$$
$$\mathsf{H} = \mathsf{Th}_2 : \{0,1\}^N \times \{0,1\}^{256} \times \{0,1\}^{2N} \to \{0,1\}^N,$$
$$\mathsf{T}_\ell : \{0,1\}^N \times \{0,1\}^{256} \times \{0,1\}^{\ell N} \to \{0,1\}^N.$$

2.2 Security Notions for Tweakable Hash Functions

We work in the post-quantum setting: adversaries have access to a quantum computer but oracles implementing secretly-keyed functions only allow classical access. All definitions are parameterized by the number of targets p and require distinct tweaks. We omit the "post-quantum" prefix for brevity.

Definition 2.1 (SM-TCR)**.** *Single-function multi-target collision resistance for distinct tweaks is defined as follows. Let* Th *be a tweakable hash function. The success probability of adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ *with* p *targets is:*

$$\mathbf{Succ}^{\mathsf{sm\text{-}tcr}}_{\mathsf{Th},p}(\mathcal{A}) = \Pr\left[\begin{array}{l} P \xleftarrow{\$} \mathcal{P};\ S \leftarrow \mathcal{A}_1^{\mathsf{Th}(P,\cdot,\cdot)}(); \\ (j, M) \leftarrow \mathcal{A}_2(Q, S, P) : \\ \mathsf{Th}(P, T_j, M_j) = \mathsf{Th}(P, T_j, M), \quad M \neq M_j, \quad \mathsf{DIST}(\{T_i\}_{i=1}^p) \end{array}\right],$$

where $\mathcal{A}_1$ *makes* p *classical queries* $Q = \{(T_i, M_i)\}_{i=1}^p$ *with distinct tweaks.*

For a THF Th_m *that is a member of a collection* $\mathbf{Th}$*,* $\mathcal{A}_1$ *additionally gets oracle access to* $\mathbf{Th}(P,\cdot,\cdot)$ *under the same* P*, with the restriction that tweaks used in the two oracles are disjoint. We write* $\mathsf{Th}_m \in \mathbf{Th}$ *in the insecurity.*

Definition 2.2 (SM-PRE)**.** *Single-function multi-target preimage resistance for distinct tweaks is defined as follows. For* Th*, parameterized by* p *targets:*

$$\mathbf{Succ}^{\mathsf{sm\text{-}pre}}_{\mathsf{Th},p}(\mathcal{A}) = \Pr\left[\begin{array}{l} P \xleftarrow{\$} \mathcal{P};\ S \leftarrow \mathcal{A}_1^{\mathsf{Th}(P,\cdot,x_i)}(); \\ (j, M) \leftarrow \mathcal{A}_2(Q, S, P) : \\ \mathsf{Th}(P, T_j, M) = \mathsf{Th}(P, T_j, x_j), \quad \mathsf{DIST}(\{T_i\}_{i=1}^p) \end{array}\right],$$

where for the i*-th query, the oracle evaluates* Th *on a uniformly random hidden* x_i*. The collection case is handled analogously.*

Definition 2.3 (SM-UD: Single-Function Multi-Target Undetectability for Distinct Tweaks)**.** *For* Th*, parameterized by* p *targets. The challenger flips a fair coin* b *and samples* $P \xleftarrow{\$} \mathcal{P}$*. Define oracle* $\mathcal{O}_P(T, b)$*: if* $b = 0$*, return* $\mathsf{Th}(P, T, x)$ *with* $x \xleftarrow{\$} \{0,1\}^N$*; if* $b = 1$*, return a uniformly random* $y \xleftarrow{\$} \{0,1\}^N$*.* $\mathcal{A}_1$ *makes* p *classical queries to* $\mathcal{O}_P(\cdot, b)$*. Then* $\mathcal{A}_2$ *receives* Q, S, P *and outputs a bit. The advantage is:*

$$\mathbf{Adv}^{\mathsf{sm\text{-}ud}}_{\mathsf{Th},p}(\mathcal{A}) = \big|\Pr[\mathcal{A}_2 \to 1 \mid b = 0 \wedge \mathsf{DIST}] - \Pr[\mathcal{A}_2 \to 1 \mid b = 1 \wedge \mathsf{DIST}]\big|.$$

Definition 2.4 (SM-DSPR: Single-Function Multi-Target Decisional Second-Preimage Resistance)**.** *The second-preimage-exists predicate is* $\mathsf{SP}_{P,T}(M) = 1$ *iff* $|\mathsf{Th}^{-1}_{P,T}(\mathsf{Th}_{P,T}(M))| \geq 2$*. For adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ *with* p *targets:*

$$\mathbf{Adv}^{\mathsf{sm\text{-}dspr}}_{\mathsf{Th},p}(\mathcal{A}) = \max\{0,\ \mathsf{succ} - \mathsf{triv}\},$$

where

$$\mathsf{succ} = \Pr[P \xleftarrow{\$} \mathcal{P}; S \leftarrow \mathcal{A}_1^{\mathsf{Th}(P,\cdot,\cdot)}(); (j, b) \leftarrow \mathcal{A}_2(Q, S, P) : \mathsf{SP}_{P,T_j}(M_j) = b \wedge \mathsf{DIST}],$$
$$\mathsf{triv} = \Pr[P \xleftarrow{\$} \mathcal{P}; S \leftarrow \mathcal{A}_1^{\mathsf{Th}(P,\cdot,\cdot)}(); (j, b) \leftarrow \mathcal{A}_2(Q, S, P) : \mathsf{SP}_{P,T_j}(M_j) = 1 \wedge \mathsf{DIST}].$$

2.3 Pseudorandom Functions

We use two PRFs: $\mathsf{PRF} : \{0,1\}^N \times \{0,1\}^{256} \to \{0,1\}^N$ for key generation and $\mathsf{PRF}_{\mathsf{msg}} : \{0,1\}^N \times \{0,1\}^* \to \{0,1\}^N$ for message randomization. Standard PRF security (pq-prf) is assumed; see [1].

2.4 Interleaved Target Subset Resilience

For the message digest function $\mathsf{H}_{\mathsf{msg}} : \{0,1\}^N \times \{0,1\}^* \to \{0,1\}^{8m}$, where m is the digest length in bytes, we require interleaved target subset resilience (ITSR) as defined in [2, Definition 18] and [1]. We refer to the standard definition and use $\mathbf{InSec}^{\mathsf{itsr}}(\mathsf{H}_{\mathsf{msg}}; \tau, q)$ for the insecurity against q-query adversaries.

2.5 EU-naCMA Security Model

Definition 2.5 (EU-naCMA)**.** *Let* $\mathsf{Dss} = (\mathsf{Kg}, \mathsf{Sign}, \mathsf{Vf})$ *be a digital signature scheme. The EU-naCMA experiment with two-stage adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ *is:*

1. $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{Kg}(1^N)$.
2. $(\{M_1, \ldots, M_q\}, S) \leftarrow \mathcal{A}_1()$. *(Non-adaptive: messages chosen before seeing* pk*.)*
3. *Compute* $\sigma_i \leftarrow \mathsf{Sign}(M_i, \mathsf{sk})$ *for* $i \in [q]$.
4. $(M^*, \sigma^*) \leftarrow \mathcal{A}_2(S, \{\sigma_i\}_{i=1}^q, \mathsf{pk})$.
5. *Return* 1 *iff* $\mathsf{Vf}(\mathsf{pk}, \sigma^*, M^*) = 1$ *and* $M^* \notin \{M_i\}_{i=1}^q$.

For one-time signatures, we set $q = 1$.

3 CS-WOTS+: Constant Sum WOTS+

3.1 The Poset and Antichain

Let $w \geq 2$ be the Winternitz parameter and $\ell \in \mathbb{N}$ be the number of chains. Consider the poset $([w]^\ell, \preceq)$ where $[w] = \{0, 1, \ldots, w-1\}$ and $\mathbf{a} \preceq \mathbf{b}$ iff $a_i \leq b_i$ for all $i \in [\ell]$.

Definition 3.1 (Constant-sum sets)**.** *For* $s \in \{0, 1, \ldots, \ell(w-1)\}$*, define*

$$\mathcal{C}_s = \left\{ \mathbf{v} \in [w]^\ell : \sum_{i=1}^{\ell} v_i = s \right\}.$$

Lemma 3.2 (Antichain property)**.** *For any* s*,* $\mathcal{C}_s$ *is an antichain: for any distinct* $\mathbf{a}, \mathbf{b} \in \mathcal{C}_s$*, neither* $\mathbf{a} \preceq \mathbf{b}$ *nor* $\mathbf{b} \preceq \mathbf{a}$.

Proof. Assume $\mathbf{a} \preceq \mathbf{b}$ and $\mathbf{a} \neq \mathbf{b}$. Then $\exists j : a_j < b_j$ and $\forall i : a_i \leq b_i$, so $\sum a_i < \sum b_i$, contradicting $\sum a_i = \sum b_i = s$. The symmetric argument excludes $\mathbf{b} \preceq \mathbf{a}$. □

Lemma 3.3 (Maximal antichain)**.** $\mathcal{C}_{s^*}$ *with* $s^* = \lfloor \ell(w-1)/2 \rfloor$ *is a maximum antichain of* $([w]^\ell, \preceq)$*. Moreover,* $|\mathcal{C}_{s^*}| = \max_s |\mathcal{C}_s|$.

Proof. It is enough, by Dilworth's theorem [5], to exhibit a chain decomposition of $([w]^\ell, \preceq)$ with exactly $|\mathcal{C}_{s^*}|$ chains. We use the standard symmetric-chain argument for a product of chains, specialized to the present notation.

For $r \geq 1$, let $S_r = ([w]^r, \preceq)$. We construct, by induction on r, a chain decomposition of S_r such that each chain $C = \{\mathbf{c}_1 \prec \cdots \prec \mathbf{c}_m\}$ satisfies

$$|\mathbf{c}_{i+1}| = |\mathbf{c}_i| + 1 \quad \text{and} \quad |\mathbf{c}_1| + |\mathbf{c}_m| = r(w-1),$$

where $|\mathbf{c}|$ denotes the sum of the coordinates. The case $r = 1$ is the single chain $(0) \prec (1) \prec \cdots \prec (w-1)$.

Assume such a decomposition is known for S_{r-1}. For a chain $C = \{\mathbf{c}_1, \ldots, \mathbf{c}_m\}$ in that decomposition, let $u = \min(w-1, m-1)$. Replace C by the $u+1$ chains, indexed by $j = 0, \ldots, u$,

$$(\mathbf{c}_1, j) \preceq \cdots \preceq (\mathbf{c}_{m-j}, j) \preceq (\mathbf{c}_{m-j}, j+1) \preceq \cdots \preceq (\mathbf{c}_{m-j}, w-1).$$

The final segment is omitted when $j = w-1$. These chains partition all pairs $(\mathbf{c}_i, a)$ with $\mathbf{c}_i \in C$ and $a \in \{0, \ldots, w-1\}$ exactly once. They also preserve the two displayed properties: consecutive elements increase the coordinate sum by one, and the first and last sums add to $r(w-1)$. Applying this replacement to every chain of the decomposition of S_{r-1} gives the desired decomposition of S_r.

For $r = \ell$, every symmetric chain contains exactly one vector with coordinate sum $s^* = \lfloor \ell(w-1)/2 \rfloor$. Hence the number of chains in the decomposition is $|\mathcal{C}_{s^*}|$. Since any antichain intersects each chain in at most one point, no antichain can be larger than $|\mathcal{C}_{s^*}|$. Lemma 3.2 shows that $\mathcal{C}_{s^*}$ itself is an antichain, so it is maximum. □

3.2 Cardinality and Recurrence

For $r, m \geq 0$, let $D_{r,m}$ be the number of r-tuples of $\{0, \ldots, w-1\}$ summing to m. This satisfies:

$$D_{0,0} = 1, \quad D_{0,m} = 0 \ (m \neq 0), \quad D_{r,m} = \sum_{i=0}^{w-1} D_{r-1,m-i}, \tag{1}$$

with $D_{r,m} = 0$ for $m < 0$ or $m > r(w-1)$.

The CS-WOTS+ parameter ℓ is chosen as the minimal integer such that

$$D_{\ell, \lfloor \ell(w-1)/2 \rfloor} \geq 2^N. \tag{2}$$

3.3 Encoding and Decoding

Let $C_{\max} = |\mathcal{C}_{s^*}| = D_{\ell,s^*}$ and assume $C_{\max} \geq 2^N$. We define a bijection $\mathsf{Enc} : [0, C_{\max} - 1] \to \mathcal{C}_{s^*}$ that maps an integer to its rank in the antichain, using the combinatorial numbering system based on Eq. (1). The inverse is $\mathsf{Dec} : \mathcal{C}_{s^*} \to [0, C_{\max} - 1]$.

For an N-bit message $m \in \{0,1\}^N$, the encoding is:

$$\mathsf{cs\text{-}wots\text{-}encode}(m) = \mathsf{Enc}(\mathsf{toInt}(m)),$$

with $\mathsf{toInt}(m) \in [0, 2^N - 1] \subseteq [0, C_{\max} - 1]$.

Lemma 3.4 (Unforgeability of CS-WOTS+ encoding)**.** *For any distinct messages $m \neq m' \in \{0,1\}^N$, let $\mathbf{b} = \mathsf{cs\text{-}wots\text{-}encode}(m)$ and $\mathbf{b}' = \mathsf{cs\text{-}wots\text{-}encode}(m')$. Then $\mathbf{b}$ and $\mathbf{b}'$ are incomparable. In particular, there exists $j \in [\ell]$ such that $b'_j < b_j$.*

Proof. Since $\mathbf{b}, \mathbf{b}' \in \mathcal{C}_{s^*}$, Lemma 3.2 implies they are incomparable. Hence there exists j with $b'_j < b_j$ (otherwise $\mathbf{b} \preceq \mathbf{b}'$, contradiction) and also k with $b_k < b'_k$. □

Remark 3.5. *This is the fundamental property that* replaces the checksum *in the WOTS-TW proof. In standard WOTS+, modifying a message forces an increase in some m_i and a decrease in some checksum component C_j, so at least one chain index decreases. In CS-WOTS+, the antichain property directly guarantees that for any two distinct messages, there exist indices with $b'_j < b_j$ and $b'_k > b_k$, which is exactly what the reduction needs.*

3.4 CS-WOTS-TW Scheme

We now define CS-WOTS-TW, which is WOTS-TW [1, Section 3] with CS-WOTS+ encoding.

Parameters: n (security parameter in bytes, with $N = 8n$ bits), w (Winternitz parameter), ℓ (from Eq. (2)).

Chaining function: For $x \in \{0,1\}^N$, iteration count $k \geq 0$, start index j, chain index i, and public seed seed:

$$c_{j,k}(x, i, \mathsf{seed}) = \begin{cases} x, & k = 0, \\ \mathsf{Th}(\mathsf{seed}, T_{i,j+k-1}, c_{j,k-1}(x, i, \mathsf{seed})), & k > 0. \end{cases}$$

If multiple instances are used with different addresses, we write $c^{\mathsf{ADRS}}_{j,k}(x, i, \mathsf{seed})$.

Key Generation $\mathsf{CS\text{-}WOTS\text{-}TW.kg}(C = (\mathsf{seed}, \mathsf{ADRS}); S)$:

- Compute $\mathsf{sk}_i \leftarrow \mathsf{PRF}(S, T_{i,0})$ for $i \in [\ell]$, where $T_{i,j}$ are tweaks derived from ADRS via the addressing scheme.
- Compute $\mathsf{pk}_i = c_{0,w-1}(\mathsf{sk}_i, i, \mathsf{seed})$.
- Return $\mathsf{sk} = (S, C)$ and $\mathsf{pk} = (\mathsf{pk} = (\mathsf{pk}_1, \ldots, \mathsf{pk}_\ell), C)$.

Signature $\mathsf{CS\text{-}WOTS\text{-}TW.sign}(M, \mathsf{sk})$:

- Compute $\mathbf{b} = (b_1, \ldots, b_\ell) = \mathsf{cs\text{-}wots\text{-}encode}(M)$.
- Compute $\sigma_i = c_{0,b_i}(\mathsf{sk}_i, i, \mathsf{seed})$ for $i \in [\ell]$.
- Return $\sigma = (\sigma_1, \ldots, \sigma_\ell)$.

Verification $\mathsf{CS\text{-}WOTS\text{-}TW.vf}(M, \sigma, \mathsf{pk})$:

- Compute $\mathbf{b}$ as above.
- Check whether $\mathsf{pk}_i = c_{b_i,w-1-b_i}(\sigma_i, i, \mathsf{seed})$ for all i. Return true if equality holds, false otherwise.

4 Tight Security of CS-WOTS-TW (Single Instance)

We prove that CS-WOTS-TW is EU-naCMA secure (one-time, i.e., $q = 1$), assuming the THF Th satisfies SM-TCR, SM-PRE, and SM-UD.

Theorem 4.1 (Security of CS-WOTS-TW, single instance)**.** *Let $n, w \in \mathbb{N}$ with $N = 8n$ and $w = \mathrm{poly}(N)$. Let $\mathsf{Th} : \mathcal{P} \times \mathcal{T} \times \{0,1\}^N \to \{0,1\}^N$ be SM-TCR, SM-PRE, and SM-UD secure. Let $\mathsf{PRF} : \mathcal{S} \times \mathcal{T} \to \{0,1\}^N$ be a PRF. Then:*

$$\begin{aligned} \mathbf{InSec}^{\mathsf{EU\text{-}naCMA}}(\mathsf{CS\text{-}WOTS\text{-}TW}; \tau, 1) \leq\ & \mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}; \tau, \ell) + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{Th}; \tau, \ell w) \\ & + \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathsf{Th}; \tau, \ell) + w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{Th}; \tau, \ell), \quad (3) \end{aligned}$$

with $\tau' = \tau + \ell w$ (time measured in Th evaluations).

Proof. We proceed via a sequence of four games. Let $\mathcal{A}$ be an EU-naCMA adversary for CS-WOTS-TW.

GAME.1 (EU-naCMA game)

This is the original EU-naCMA experiment (Definition 2.5). $\mathcal{A}_1$ outputs a message M. The experiment generates $(\mathsf{sk}, \mathsf{pk}) = \mathsf{CS\text{-}WOTS\text{-}TW.kg}(C = (\mathsf{seed}, \mathsf{ADRS}); S)$ with $\mathsf{seed} \xleftarrow{\$} \{0,1\}^N$, $S \xleftarrow{\$} \{0,1\}^N$, computes $\sigma \leftarrow \mathsf{CS\text{-}WOTS\text{-}TW.sign}(M, \mathsf{sk})$, and returns (σ, pk) to $\mathcal{A}_2$. $\mathcal{A}_2$ wins if it outputs a valid forgery (M', σ') with $M' \neq M$.

GAME.2 (Replace PRF with random function)

Same as GAME.1 but all outputs of $\mathsf{PRF}(S, \cdot)$ are replaced by independent uniformly random values. Since all ℓ tweaks $T_{i,0}$ are distinct, the PRF outputs are independent. The difference is bounded by the PRF advantage.

Claim 4.2. $|\mathbf{Succ}_{\mathbf{GAME}.1}(\mathcal{A}) - \mathbf{Succ}_{\mathbf{GAME}.2}(\mathcal{A})| \leq \mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}; \tau, \ell)$.

Proof. Standard reduction: the distinguisher uses its PRF oracle to simulate GAME.1 (oracle is PRF) or GAME.2 (oracle is random), making ℓ distinct queries. The difference in success probability equals the PRF distinguishing advantage. □

GAME.3 (Replace chain values with hashes of random values)

In this game, upon receiving $\mathcal{A}_1$'s signing query M (without knowing seed), we construct the signature differently. Let $\mathbf{b} = \mathsf{cs\text{-}wots\text{-}encode}(M)$. Generate ℓ random values $u_i \xleftarrow{\$} \{0,1\}^N$, $i \in [\ell]$. Set the signature as

$$\sigma_i = \begin{cases} \mathsf{Th}(\mathsf{seed}, T_{i,b_i-1}, u_i), & b_i > 0, \\ u_i, & b_i = 0. \end{cases}$$

Then request seed and compute the public key via $\mathsf{pk}_i = c_{b_i, w-1-b_i}(\sigma_i, i, \mathsf{seed})$, exactly as in verification. Return $(\sigma, \mathsf{pk} = (\mathsf{pk}_1, \ldots, \mathsf{pk}_\ell), C)$.

Claim 4.3. $|\mathbf{Succ}_{\mathbf{GAME}.2}(\mathcal{A}) - \mathbf{Succ}_{\mathbf{GAME}.3}(\mathcal{A})| \leq w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{Th}; \tau, \ell)$.

Proof. Let $\mathbf{b} = (b_1, \ldots, b_\ell)$. Define two distributions:

$$D_{\mathsf{Kg}} = \{\, y_i = c_{0,b_i-1}(\tilde{u}_i, i, \mathsf{seed}) : \tilde{u}_i \xleftarrow{\$} \{0,1\}^N,\ i \in [\ell] \,\},$$
$$D_0 = \{\, \varepsilon_i \xleftarrow{\$} \{0,1\}^N : i \in [\ell] \,\}.$$

Samples from D_{Kg} are distributed as the (b_i-1)-th values of valid CS-WOTS-TW chains. Samples from D_0 are uniform random.

An algorithm $\mathcal{M}^{\mathcal{A}}_{2\to 3}$ that distinguishes D_{Kg} from D_0 simulates the EU-naCMA game as follows. It receives $\mathbf{b}$ from $\mathcal{A}_1$, queries its distribution oracle $O(\mathbf{b})$ to obtain samples ψ_i, and sets

$$\sigma_i = \begin{cases} \mathsf{Th}(\mathsf{seed}, T_{i,b_i-1}, \psi_i), & b_i > 0, \\ \psi_i, & b_i = 0. \end{cases}$$

It then builds pk and returns to $\mathcal{A}_2$. When O samples from D_{Kg}, the view is GAME.2; from D_0, the view is GAME.3. Thus

$$\mathbf{Adv}_{D_0, D_{\mathsf{Kg}}}(\mathcal{M}^{\mathcal{A}}_{2\to 3}) = |\mathbf{Succ}_{\mathbf{GAME}.2}(\mathcal{A}) - \mathbf{Succ}_{\mathbf{GAME}.3}(\mathcal{A})|.$$

Let $b_{\max} = \max\{b_1, \ldots, b_\ell\} \leq w - 1$. Define intermediate hybrids H_k by

$$\psi_i^{(k)} = c_{k,b_i-1-k}(\tilde{u}_i, i, \mathsf{seed}).$$

Then $H_0 = D_{\mathsf{Kg}}$ and $H_{b_{\max}-1} = D_0$. By the hybrid argument, there exist consecutive $j, j+1$ distinguishable with advantage at least

$$\frac{\mathbf{Adv}_{D_0, D_{\mathsf{Kg}}}(\mathcal{M}^{\mathcal{A}}_{2\to 3})}{b_{\max} - 1}.$$

We now reduce distinguishing H_j from H_{j+1} to SM-UD. The reduction $\mathcal{M}^{\mathsf{UD}}$ makes ℓ queries to the SM-UD oracle $\mathcal{O}^{\mathsf{UD}}(\cdot, b)$ with tweaks $\{T_{1,j}, \ldots, T_{\ell,j}\}$. If $b = 0$, responses are $\mathsf{Th}(\mathsf{seed}, T_{i,j}, \tilde{u}_i) = c_{j,1}(\tilde{u}_i, i, \mathsf{seed})$ (random preimages, yielding hash outputs); if $b = 1$, responses are uniformly random. After obtaining seed, $\mathcal{M}^{\mathsf{UD}}$ applies the remaining chain steps $c_{j+1,b_i-1-(j+1)}$ to construct H_{j+b} and returns the forger's output. Hence

$$\mathbf{Adv}_{H_j, H_{j+1}} \le \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{Th}; \tau, \ell).$$

Combining,

$$|\mathbf{Succ}_{\mathbf{GAME}.2} - \mathbf{Succ}_{\mathbf{GAME}.3}| \le (b_{\max} - 1) \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{Th}; \tau, \ell) \le w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{Th}; \tau, \ell).$$

□

GAME.4 (Exclude chain collisions)

GAME.4 is the same as GAME.3, but the game is declared lost if $\mathcal{A}$ outputs a valid forgery (M', σ') for which there exists $i \in [\ell]$ such that $b'_i < b_i$ and

$$c_{b'_i, b_i - b'_i}(\sigma'_i, i, \mathsf{seed}) = \sigma_i.$$

(Here $\mathbf{b}$ is from the queried message M, and $\mathbf{b}'$ from the forgery M'.)

Claim 4.4. $|\mathbf{Succ}_{\mathbf{GAME}.3}(\mathcal{A}) - \mathbf{Succ}_{\mathbf{GAME}.4}(\mathcal{A})| \le \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{Th}; \tau, \ell w)$.

Proof. We construct an SM-TCR adversary $\mathcal{M}^{\mathsf{TCR}}$ that simulates GAME.3. Upon receiving $\mathcal{A}_1$'s message M, $\mathcal{M}^{\mathsf{TCR}}$ encodes it as $\mathbf{b}$ and, for each i with $b_i > 0$, samples a random $\tilde{u}_i$ and queries the SM-TCR oracle with $(T_{i,b_i-1}, \tilde{u}_i)$ to obtain a target $\sigma_i = \mathsf{Th}(P, T_{i,b_i-1}, \tilde{u}_i)$. Then it continues querying the SM-TCR oracle to build the rest of each chain: $(T_{i,b_i}, \sigma_i) \to c_1$, $(T_{i,b_i+1}, c_1) \to c_2$, $\ldots$, up to c_{w-1-b_i}, which becomes pk_i. The entire chain from position b_i to $w-1$ is built using SM-TCR challenges.

When $\mathcal{A}$ returns a forgery satisfying $c_{b'_i, b_i - b'_i}(\sigma'_i, i, \mathsf{seed}) = \sigma_i$ for some i with $b'_i < b_i$, by a pigeonhole argument there must be a collision between the honestly computed chain and the adversary's chain somewhere between index b'_i and $w-1$. $\mathcal{M}^{\mathsf{TCR}}$ extracts the first position where they diverge and collide, which yields a valid SM-TCR solution. The number of SM-TCR challenges placed is at most ℓw (covering all chain positions from b_i to $w-1$ for each i).

The simulation is perfect: the signatures given to $\mathcal{A}$ are identically distributed to GAME.3 (they are $\mathsf{Th}(\mathsf{seed}, T_{i,b_i-1}, \tilde{u}_i)$, exactly as in GAME.3). Hence the bound follows. □

Bounding GAME.4 via SM-PRE

In GAME.4, the only remaining way for $\mathcal{A}$ to win is to produce a forgery (M', σ') where for *all* i with $b'_i < b_i$, we have $c_{b'_i, b_i - b'_i}(\sigma'_i, i, \mathsf{seed}) \neq \sigma_i$. By Lemma 3.4 (the antichain property), since $M' \neq M$, there exists at least one j with $b'_j < b_j$. For this j, we have $c_{b'_j, b_j - b'_j}(\sigma'_j, j, \mathsf{seed}) \neq \sigma_j$ by the GAME.4 exclusion. But for verification to succeed, we need $c_{b'_j, w-1-b'_j}(\sigma'_j, j, \mathsf{seed}) = \mathsf{pk}_j = c_{b_j, w-1-b_j}(\sigma_j, j, \mathsf{seed})$. Since the chains from b'_j and b_j converge to the same pk_j but diverge at some intermediate point (by the exclusion), we can extract a preimage.

Claim 4.5. $\mathbf{Succ}_{\mathbf{GAME}.4}(\mathcal{A}) \le \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathsf{Th}; \tau, \ell)$.

Proof. We construct an SM-PRE adversary $\mathcal{M}^{\mathsf{PRE}}$. Upon receiving M, $\mathcal{M}^{\mathsf{PRE}}$ encodes it as $\mathbf{b}$. For each i with $b_i > 0$, it queries the SM-PRE challenger with tweak T_{i,b_i-1} to obtain a preimage challenge $y_i = \mathsf{Th}(P, T_{i,b_i-1}, x_i)$ (where x_i is hidden). It sets $\sigma_i = y_i$ as the signature element. For $b_i = 0$, it sets σ_i to a random value. Then it receives $P = \mathsf{seed}$ from the challenger and computes $\mathsf{pk}_i = c_{b_i,w-1-b_i}(\sigma_i, i, \mathsf{seed})$ by finishing the chains. The resulting (σ, pk) has exactly the GAME.3/GAME.4 distribution.

When $\mathcal{A}$ returns a forgery, Lemma 3.4 gives an index j with $b'_j < b_j$. In GAME.4,

$$c_{b'_j,b_j-b'_j}(\sigma'_j, j, \mathsf{seed}) \neq \sigma_j.$$

The SM-PRE challenge $y_j = \sigma_j$ was computed at position $b_j - 1$ as $\mathsf{Th}(P, T_{j,b_j-1}, x_j)$. The preimage of σ_j under $\mathsf{Th}(P, T_{j,b_j-1}, \cdot)$ provided by the forgery path is

$$c_{b'_j,b_j-b'_j-1}(\sigma'_j, j, \mathsf{seed}).$$

It must differ from x_j, since otherwise the chain would already converge at $b_j - 1$. Therefore $\mathcal{M}^{\mathsf{PRE}}$ extracts a valid preimage and wins the SM-PRE game whenever $\mathcal{A}$ succeeds.

The simulation uses ℓ distinct SM-PRE challenges, and the reduction's success probability equals $\mathbf{Succ}_{\mathbf{GAME}.4}(\mathcal{A})$. □

Combining Claims 1–4 via the triangle inequality yields the theorem bound. □

5 Multi-Instance CS-WOTS-TW

In SPHINCS+/Kylin, many CS-WOTS-TW instances are used within one key pair, all sharing the same public seed seed but distinguished by different address prefixes ADRS. We follow [1, Section 6] and extend the EU-naCMA model to D instances. This avoids overloading the Kylin parameter d, which denotes the number of hypertree layers.

Theorem 5.1 (Security of CS-WOTS-TW, multi-instance)**.** *Let n, w, ℓ be as above, with $N = 8n$. Let $\mathsf{F} = \mathsf{Th}_1 : \mathcal{P} \times \mathcal{T} \times \{0,1\}^N \to \{0,1\}^N$ be SM-TCR, SM-PRE, and SM-UD secure as a member of a collection $\mathbf{Th}$. Let $\mathsf{PRF} : \mathcal{S} \times \mathcal{T} \to \{0,1\}^N$ be a PRF. Then for any adversary making one signature query per instance (i.e., D queries total):*

$$\begin{aligned}\mathbf{InSec}^{\mathsf{D\text{-}EU\text{-}naCMA}}(\mathsf{CS\text{-}WOTS\text{-}TW}; \tau, D) \;<\; & \mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}; \tau, D{\cdot}\ell) \\ & + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{F} \in \mathbf{Th}; \tau, D{\cdot}\ell w) \\ & + \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathsf{F} \in \mathbf{Th}; \tau, D{\cdot}\ell) + w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{F} \in \mathbf{Th}; \tau, D{\cdot}\ell),\end{aligned}$$

with $\tau' = \tau + D \cdot \ell w$.

Proof sketch. The proof follows the same four-game sequence as Theorem 4.1, with the following adjustments for D instances:

GAME.1 → GAME.2: The PRF is replaced by random values for all $D \cdot \ell$ secret key elements. Since each instance uses a unique ADRS prefix, all tweaks are distinct, so all PRF outputs are independent random values. The bound becomes $\mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}; \tau, D{\cdot}\ell)$.

GAME.2 → GAME.3 (UD hybrid): The two multi-instance distributions now contain $D{\cdot}\ell$ values, one per chain across all D instances. The hybrid argument uses $b_{\max} \leq w - 1$ as before but with $D \cdot \ell$ values per hybrid. The SM-UD reduction queries $D\ell$ targets per hybrid step. The bound is $w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{F} \in \mathbf{Th}; \tau, D{\cdot}\ell)$.

GAME.3 → GAME.4 (TCR exclusion): The SM-TCR reduction places challenges across all $D \cdot \ell w$ chain positions. The bound is $\mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{F} \in \mathbf{Th}; \tau, D{\cdot}\ell w)$.

GAME.4 bound (PRE extraction): The SM-PRE reduction places $D\ell$ preimage challenges. If the adversary forges for instance r, the antichain property (Lemma 3.4) guarantees $\exists j$ with

$b'_{r,j} < b_{r,j}$, and the PRE extraction works exactly as in the single-instance case. The bound is $\mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathsf{F} \in \mathbf{Th}; \tau, D \cdot \ell)$.

For the collection aspect: during SM-UD, SM-TCR, and SM-PRE reductions, all THF evaluations for tweaks that are not part of the challenge are simulated using the collection oracle $\mathbf{Th}(P, \cdot, \cdot)$. The disjointness of challenge and collection tweaks is guaranteed by the addressing scheme. □

6 The Kylin Construction

6.1 Parameters

Kylin is parameterized by:

- n: security parameter (bytes),
- w: Winternitz parameter,
- ℓ: CS-WOTS+ chain count (from Eq. (2)),
- h: total hypertree height,
- d: number of hypertree layers,
- $(h_0, \ldots, h_{d-1})$: per-layer heights, $\sum_{i=0}^{d-1} h_i = h$,
- k: number of FORS trees,
- $t = 2^a$: leaves per FORS tree,
- m: message digest length in bytes.

6.2 Hypertree with Diverse Layer Heights

The hypertree consists of d layers of XMSS trees, indexed from the bottom layer 0 to the top layer $d-1$. Layer i contains

$$2^{h - \sum_{j=0}^{i} h_j}$$

XMSS trees, each of height h_i. Since $\sum_{j=0}^{d-1} h_j = h$, the top layer contains a single XMSS tree.

Each XMSS tree of height h_i has 2^{h_i} leaves, which are CS-WOTS-TW public keys. Internal nodes are computed using $\mathsf{H} = \mathsf{Th}_2$. The root of each tree is signed by the CS-WOTS-TW instance of the layer above. The bottom layer's CS-WOTS-TW instances sign FORS public keys.

The HT public key is the root of the unique XMSS tree at layer $d-1$. An HT signature on message M at position $(\mathsf{idx}_{\mathsf{tree}}, \mathsf{idx}_{\mathsf{leaf}})$ consists of d XMSS signatures. At layer i, the XMSS signature contains one CS-WOTS-TW signature of ℓ n-byte strings and an XMSS authentication path of h_i n-byte nodes. The $\log t = a$ FORS authentication nodes belong only to σ_{FORS}, not to every HT layer.

6.3 FORS and Full Kylin

FORS (Forest of Random Subsets) is identical to SPHINCS+ [2]. The full scheme is:

- kylin_keygen: sample

$$\mathsf{sk.seed},\ \mathsf{sk.PRF},\ \mathsf{pk.seed} \xleftarrow{\$} \{0,1\}^N,$$

compute $\mathsf{pk.root} = \mathsf{ht_PKgen}(\mathsf{sk.seed}, \mathsf{pk.seed})$, and return

$$\mathsf{sk} = (\mathsf{sk.seed}, \mathsf{sk.PRF}, \mathsf{pk.seed}, \mathsf{pk.root}), \qquad \mathsf{pk} = (\mathsf{pk.seed}, \mathsf{pk.root}).$$

- $\mathsf{kylin_sign}(M, \mathsf{sk})$:
  1. $R \leftarrow \mathsf{PRF}_{\mathsf{msg}}(\mathsf{sk.PRF}, \mathsf{opt}, M)$.
  2. $(\mathsf{md}, \mathsf{idx}_{\mathsf{tree}}, \mathsf{idx}_{\mathsf{leaf}}) \leftarrow \mathsf{H}_{\mathsf{msg}}(R, \mathsf{pk.seed}, \mathsf{pk.root}, M)$.
  3. $\sigma_{\mathsf{FORS}} \leftarrow \mathsf{fors_sign}(\mathsf{md}, \mathsf{sk.seed}, \mathsf{pk.seed}, \mathsf{ADRS}_{\mathsf{FORS}}(\mathsf{idx}_{\mathsf{tree}}, \mathsf{idx}_{\mathsf{leaf}}))$.
  4. $\mathsf{pk}_{\mathsf{FORS}} \leftarrow \mathsf{fors_pkFromSig}(\sigma_{\mathsf{FORS}}, \mathsf{md}, \mathsf{pk.seed}, \mathsf{ADRS}_{\mathsf{FORS}})$.
  5. $\sigma_{\mathsf{HT}} \leftarrow \mathsf{ht_sign}(\mathsf{pk}_{\mathsf{FORS}}, \mathsf{sk.seed}, \mathsf{pk.seed}, \mathsf{idx}_{\mathsf{tree}}, \mathsf{idx}_{\mathsf{leaf}})$.
  6. Return $\sigma = (R, \sigma_{\mathsf{FORS}}, \sigma_{\mathsf{HT}})$.
- $\mathsf{kylin_verify}(M, \sigma, \mathsf{pk})$: Recompute digest and indices; recover $\mathsf{pk}_{\mathsf{FORS}}$ from σ_{FORS}; verify $\mathsf{ht_verify}(\mathsf{pk}_{\mathsf{FORS}}, \sigma_{\mathsf{HT}}, \mathsf{pk.seed}, \mathsf{idx}_{\mathsf{tree}}, \mathsf{idx}_{\mathsf{leaf}}, \mathsf{pk.root})$.

6.4 Signature Size

With diverse layer heights, the signature size is

$$|\sigma| = \big(1 + k(a+1) + h + d \cdot \ell\big)\, n \text{ bytes.}$$

Note that $h = \sum h_i$, so the total HT authentication-path size remains $h \cdot n$ bytes (equivalently, hN bits) regardless of the distribution across layers.

7 Security Reduction for Kylin

Theorem 7.1 (EUF-CMA security of Kylin)**.** *For parameters $n, w, h, d, (h_0, \ldots, h_{d-1}), m, t, k, \ell$ as described, Kylin is EUF-CMA secure if the underlying primitives satisfy the required properties. Concretely, for any adversary $\mathcal{A}$ making q_s signing queries:*

$$\begin{aligned}
\mathbf{InSec}^{\mathsf{EU\text{-}CMA}}(\mathsf{Kylin}; \tau, q_s) \;\le\; & \mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}; \tau, q_1) + \mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}_{\mathsf{msg}}; \tau, q_s) \\
& + \mathbf{InSec}^{\mathsf{itsr}}(\mathsf{H}_{\mathsf{msg}}; \tau, q_s) \\
& + w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{F} \in \mathbf{Th}; \tau, q_2) \\
& + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{F} \in \mathbf{Th}; \tau, q_3 + q_7) + \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathsf{F} \in \mathbf{Th}; \tau, q_2) \\
& + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{H} \in \mathbf{Th}; \tau, q_4) + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{T}_k \in \mathbf{Th}; \tau, q_5) \\
& + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{T}_\ell \in \mathbf{Th}; \tau, q_6) + 3 \cdot \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{F} \in \mathbf{Th}; \tau, q_8) \\
& + \mathbf{InSec}^{\mathsf{sm\text{-}dspr}}(\mathsf{F} \in \mathbf{Th}; \tau, q_8), \qquad (4)
\end{aligned}$$

where

$$q_1 < 2^{h+1}(kt + \ell), \quad q_2 < 2^{h+1} \cdot \ell, \quad q_3 < 2^{h+1} \cdot \ell w, \quad q_4 < 2^{h+1} k \cdot 2t,$$
$$q_5 < 2^h, \quad q_6 < 2^{h+1}, \quad q_7 < 2^{h+1} kt, \quad q_8 < 2^h \cdot kt.$$

Proof. We adapt the game sequence of [1, Theorem 3], which is in turn based on [2, Theorem 17]. The bound for GAME.0–GAME.3 is identical to the SPHINCS+ proof and is only sketched; the main novelty is in GAME.3.4 → GAME.4, where we invoke the CS-WOTS-TW multi-instance bound (Theorem 5.1).

GAME.0: The EUF-CMA Experiment

This is the standard EUF-CMA game for Kylin. The adversary receives a real public key and access to a signing oracle.

GAME.0 → GAME.1: Replace PRF

All outputs of $\mathsf{PRF}(\mathsf{sk.seed}, \cdot)$ (used to generate CS-WOTS-TW and FORS secret key elements) are replaced by independent uniformly random values. By the same argument as in [1, Eq. (4)]:

$$|\mathbf{Succ}_{\mathbf{GAME}.0}(\mathcal{A}) - \mathbf{Succ}_{\mathbf{GAME}.1}(\mathcal{A})| \leq \mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}; \tau, q_1),$$

with $q_1 < 2^{h+1}(kt + \ell)$ (accounting for all FORS instances and all CS-WOTS-TW instances across the hypertree).

GAME.1 → GAME.2: Replace PRFmsg

$\mathsf{PRF}_{\mathsf{msg}}(\mathsf{sk.PRF}, \cdot)$ is replaced by a truly random function. As in [1, Eq. (5)]:

$$|\mathbf{Succ}_{\mathbf{GAME}.1}(\mathcal{A}) - \mathbf{Succ}_{\mathbf{GAME}.2}(\mathcal{A})| \leq \mathbf{InSec}^{\mathsf{prf}}(\mathsf{PRF}_{\mathsf{msg}}; \tau, q_s).$$

GAME.2 → GAME.3: ITSR

The game is declared lost if the adversary outputs a forgery where the FORS signature contains only secret values previously revealed for the same FORS instance. As in [1, Eq. (6)]:

$$|\mathbf{Succ}_{\mathbf{GAME}.2}(\mathcal{A}) - \mathbf{Succ}_{\mathbf{GAME}.3}(\mathcal{A})| \leq \mathbf{InSec}^{\mathsf{itsr}}(\mathsf{H}_{\mathsf{msg}}; \tau, q_s).$$

GAME.3: Decomposition into Sub-Cases

In GAME.3, any successful forgery must contain a FORS signature with at least one fresh leaf value. We decompose the remaining adversarial advantage into sub-cases based on where in the verification structure a collision or forgery occurs. Following [1], we define intermediate games:

GAME.3.1 (SM-TCR for H, binary tree collisions) The game is lost if a second preimage for an input to H is contained in the forged signature, occurring in any binary tree (Merkle tree) within the hypertree or FORS trees. As in [1, Eq. (7)]:

$$|\mathbf{Succ}_{\mathbf{GAME}.3} - \mathbf{Succ}_{\mathbf{GAME}.3.1}| \leq \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{H} \in \mathbf{Th}; \tau, q_4),$$

with $q_4 < 2^{h+1}k \cdot 2t$.

GAME.3.2 (SM-TCR for T_k, FORS root compression) The game is lost if a second preimage for T_k (compressing k FORS roots) is found. As in [1, Eq. (8)]:

$$|\mathbf{Succ}_{\mathbf{GAME}.3.1} - \mathbf{Succ}_{\mathbf{GAME}.3.2}| \leq \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{T}_k \in \mathbf{Th}; \tau, q_5),$$

with $q_5 < 2^h$.

GAME.3.3 (SM-TCR for T_ℓ, CS-WOTS+ public key compression) The game is lost if a second preimage for T_ℓ (compressing ℓ CS-WOTS+ chain heads) is found. As in [1, Eq. (9)]:

$$|\mathbf{Succ}_{\mathbf{GAME}.3.2} - \mathbf{Succ}_{\mathbf{GAME}.3.3}| \leq \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{T}_\ell \in \mathbf{Th}; \tau, q_6),$$

with $q_6 < 2^{h+1}$.

GAME.3.4 (SM-TCR for F, FORS leaf values) The game is lost if a second preimage for F is found outside the CS-WOTS-TW instances, i.e., in FORS leaf computations. As in [1, Eq. (10)]:

$$|\mathbf{Succ}_{\mathbf{GAME}.3.3} - \mathbf{Succ}_{\mathbf{GAME}.3.4}| \leq \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{F} \in \mathbf{Th}; \tau, q_7),$$

with $q_7 < 2^{h+1}kt$.

GAME.3.4 → GAME.4 (CS-WOTS-TW multi-instance forgery) After excluding all the above collision cases, the only remaining way to forge is to produce a valid CS-WOTS-TW signature for one of the fewer than 2^{h+1} virtual instances in the hypertree. Applying Theorem 5.1:

$$|\mathbf{Succ}_{\mathbf{GAME}.3.4} - \mathbf{Succ}_{\mathbf{GAME}.4}| \le \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{F} \in \mathbf{Th}; \tau, q_3) + \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathsf{F} \in \mathbf{Th}; \tau, q_2) + w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathsf{F} \in \mathbf{Th}; \tau, q_2),$$

where $q_2 < 2^{h+1}\ell$ and $q_3 < 2^{h+1}\ell w$.

GAME.4: Final Bound

GAME.4 is exactly the situation analyzed in [2, Claim 23] and [1, Eq. (12)]: the adversary must forge a FORS signature where the leaf value differs from the expected one (the "second leaf case"). This reduces to SM-TCR and SM-DSPR of F:

$$\mathbf{Succ}_{\mathbf{GAME}.4}(\mathcal{A}) \le 3 \cdot \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathsf{F} \in \mathbf{Th}; \tau, q_8) + \mathbf{InSec}^{\mathsf{sm\text{-}dspr}}(\mathsf{F} \in \mathbf{Th}; \tau, q_8),$$

with $q_8 < 2^h \cdot kt$.

Assembling all game transitions via the triangle inequality yields Eq. (4), completing the proof. □

Remark 7.2. *The diverse layer heights $(h_0, \ldots, h_{d-1})$ do not affect the security bound. The reduction uses the total height h to bound the number of virtual leaves, virtual CS-WOTS-TW instances, and authentication-path targets. The per-layer distribution affects concrete performance through the tree-generation costs $\sum_i 2^{h_i}$, but it does not change the adversarial advantage terms in Eq.* (4).

8 Concrete Security Against Generic Attacks

Following [1, Section 8] and [2, Section 5], the tight reduction above is instantiated with generic attack estimates for random-function-like primitives. This concrete section is not a replacement for Theorem 7.1; it is the parameter-selection proxy used to compare Kylin instances once the primitive advantages in Eq. (4) are interpreted generically.

Let q_s be the allowed number of signatures under one key, and let $t = 2^a$. The dominant Kylin/SPHINCS+-style generic forgery term is

$$p_{\mathsf{gen}} = 2^{-N} + \sum_{\gamma=0}^{q_s} \left(1 - \left(1 - \frac{1}{t}\right)^{\gamma}\right)^k \binom{q_s}{\gamma} \left(1 - \frac{1}{2^h}\right)^{q_s - \gamma} 2^{-h\gamma}.$$

Here γ is the number of previous signatures that hit the same hypertree leaf. Conditioned on such a hit count, the factor

$$\left(1 - \left(1 - \frac{1}{t}\right)^{\gamma}\right)^k$$

is the probability that each of the k FORS trees exposes a reusable index. The leading term $2^{-N} = 2^{-8n}$ accounts for the generic search probability against an n-byte primitive output.

The classical and quantum generic bit-security proxies are therefore

$$\mathsf{bitsec}_{\mathsf{cl}} = -\log_2 p_{\mathsf{gen}}, \qquad \mathsf{bitsec}_{\mathsf{q}} = -\frac{1}{2}\log_2 p_{\mathsf{gen}},$$

where small constant factors inside the logarithm are neglected, as in the Kylin parameter-selection script.

The frozen Kylin profiles in `paper/data/para_kylin_final_6profiles.csv` give the following classical exponents under this formula:

Instance	q_s	n	h	d	a	k	ℓ	$-\log_2 p_{\mathsf{gen}}$
Kylin-128	2^{64}	16	65	9	13	11	22	127.209
Kylin-256	2^{64}	32	68	10	13	22	46	255.944
Kylin-512	2^{64}	64	66	11	13	49	79	511.243
Kylin-128-q24	2^{24}	16	22	1	24	6	67	127.281
Kylin-256-q24	2^{24}	32	21	1	25	12	131	255.359
Kylin-512-q24	2^{24}	64	57	3	20	24	87	511.415

9 Comparison with the Tight SPHINCS+ Proof

We summarize the key differences between this proof and [1, Theorem 3]:

1. **CS-WOTS+ encoding.** The proof of Theorem 4.1 (GAME.4, Claim 4: Lemma 3.4) replaces the checksum argument with the antichain property (Lemma 3.2). Specifically, the sentence "By the properties of the checksum, there has to be at least one i with $b'_i < b_i$" becomes "By Lemma 3.4 (antichain property of CS-WOTS+), there exists j with $b'_j < b_j$." The rest of the SM-PRE extraction argument remains unchanged.

2. **Parameter ℓ.** Instead of $l = \lceil n/\log w \rceil + \lfloor \log(\lceil n/\log w \rceil \cdot (w-1))/\log w \rfloor + 1$, we use the minimal ℓ satisfying $D_{\ell,\lfloor \ell(w-1)/2 \rfloor} \geq 2^N$. This changes the concrete values of $q_1, \ldots, q_8$ but not the proof structure.

3. **Encoding algorithm.** CS-WOTS-TW uses cs-wots-encode based on combinatorial numbering via $D_{n,m}$ (Eq. (1)) instead of the base-w expansion with checksum. The encoding is used identically in signing and verification, so all game simulations remain valid.

4. **Diverse layer heights.** The hypertree uses per-layer heights $(h_0, \ldots, h_{d-1})$ instead of uniform h/d. Since the bounds q_1–q_8 depend only on the total height h, not its distribution, the proof remains unchanged. The only modification is in the algorithmic description of ht_sign and ht_verify where the loop over layers uses variable tree heights.

5. **SM-UD, SM-TCR, SM-PRE bounds.** All UD/TCR/PRE game transitions (Claims 2–4 of Theorem 4.1, and GAME.3.1–3.4 of Theorem 7.1) are *completely unchanged* from [1]. The SM-UD hybrid argument depends only on the chain structure ($b_{\max} \leq w-1$) and not on how b_i are derived from the message.

References

[1] Andreas Hülsing and Mikhail Kudinov. Recovering the Tight Security Proof of SPHINCS+. In *Advances in Cryptology – ASIACRYPT 2022*, LNCS 13793, pages 3–33. Springer, 2022. Full version: `https://eprint.iacr.org/2022/1018`.

[2] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS+ Signature Framework. In *ACM CCS 2019*, pages 2129–2146. ACM, 2019.

[3] Mikhail A. Kudinov, Ekaterina O. Kiktenko, and Aleksey K. Fedorov. Security Analysis of the W-OTS$^+$ Signature Scheme: Updating Security Bounds. *Mat. Vopr. Kriptogr.*, 12(2):129–145, 2021. `https://eprint.iacr.org/2020/1073`.

[4] Kylin Team. Kylin First-Stage NGCC Digital-Signature Submission Package. May 29, 2026.

[5] Robert P. Dilworth. A Decomposition Theorem for Partially Ordered Sets. *Annals of Mathematics*, 51(1):161–166, 1950.

[6] Chris Dods, Nigel P. Smart, and Martijn Stam. Hash Based Digital Signature Schemes. In *IMA Cryptography and Coding 2005*, LNCS 3796, pages 96–115. Springer, 2005.

[7] Andreas Hülsing, Joost Rijneveld, and Fang Song. Mitigating Multi-target Attacks in Hash-Based Signatures. In *PKC 2016*, LNCS 9614, pages 387–416. Springer, 2016.

[8] Daniel J. Bernstein and Andreas Hülsing. Decisional Second-Preimage Resistance: When Does SPR Imply PRE? In *ASIACRYPT 2019*, LNCS 11921, pages 33–62. Springer, 2019.

[9] Mark D. Zhandry. How to Record Quantum Queries, and Applications to Quantum Indifferentiability. In *CRYPTO 2019*, LNCS 11693, pages 239–268. Springer, 2019.

[10] Mark Zhandry. How to Construct Quantum Random Functions. In *FOCS 2012*, pages 679–687. IEEE, 2012.

[11] Eyal Kaplan, Sophie Laplante, and Frédéric Magniez. On the Power of Quantum Memory. *arXiv:quant-ph/0601121*, 2006.